

# Branches & Style Guides

*Week 2 - 06/22/2026*

## **LEARNING GOALS:**

*Learn about the causes of merge conflicts*

*Practice making branches and switching between them*

*Resolve merge conflicts using GitHub's interface*

# What are Merge Conflicts?

# What are Merge Conflicts?

An incompatible change between two versions of a file

An update to a file that has been deleted

An update to a file that has been renamed

⋮

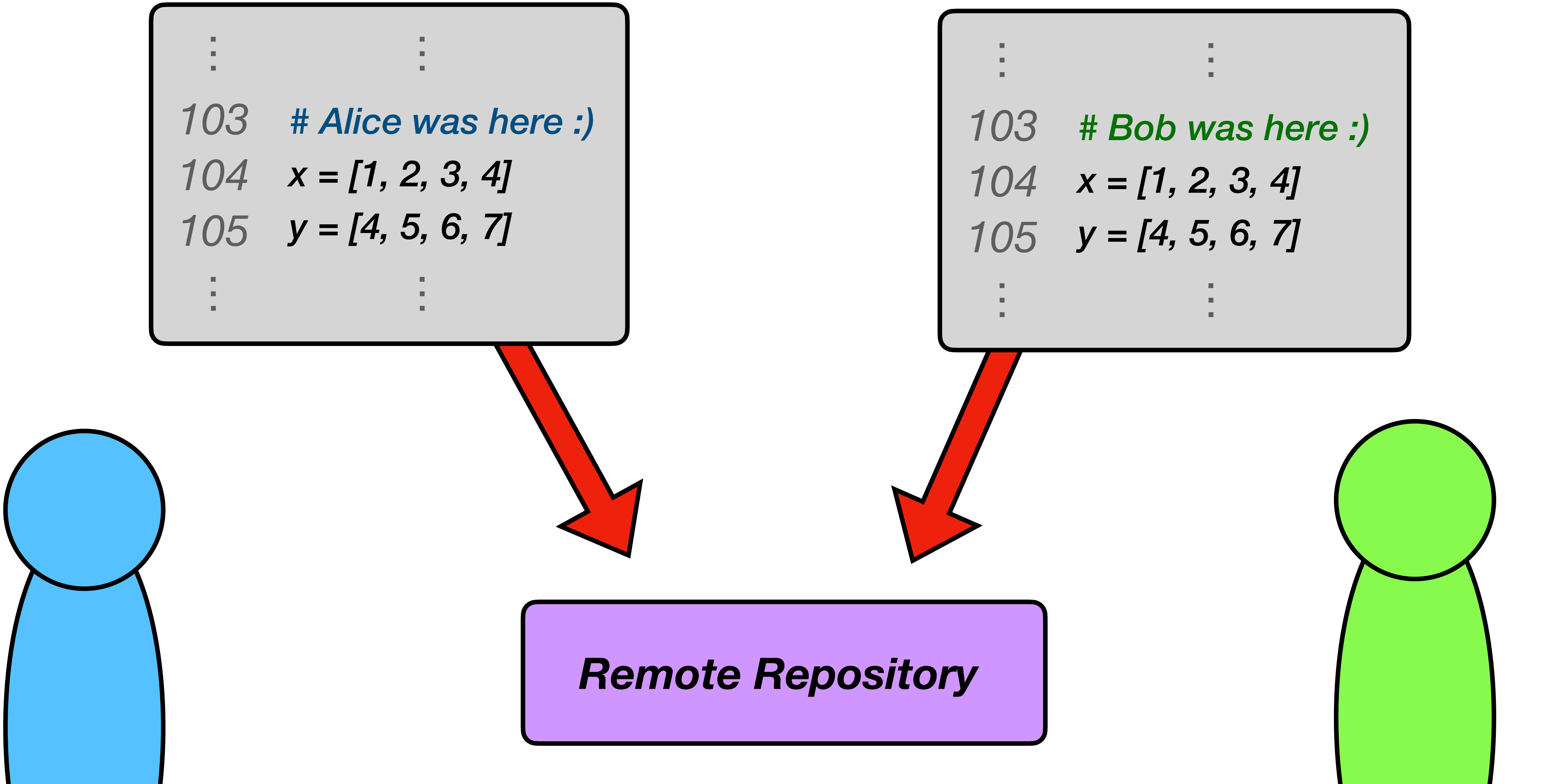
Any situation where Git cannot figure out which change to keep!



*If two coders change the same line of code, which one should be kept?*

```
⋮  
⋮  
103  # Alice was here :)  
104  x = [1, 2, 3, 4]  
105  y = [4, 5, 6, 7]  
⋮  
⋮
```

```
⋮  
⋮  
103  # Bob was here :)  
104  x = [1, 2, 3, 4]  
105  y = [4, 5, 6, 7]  
⋮  
⋮
```



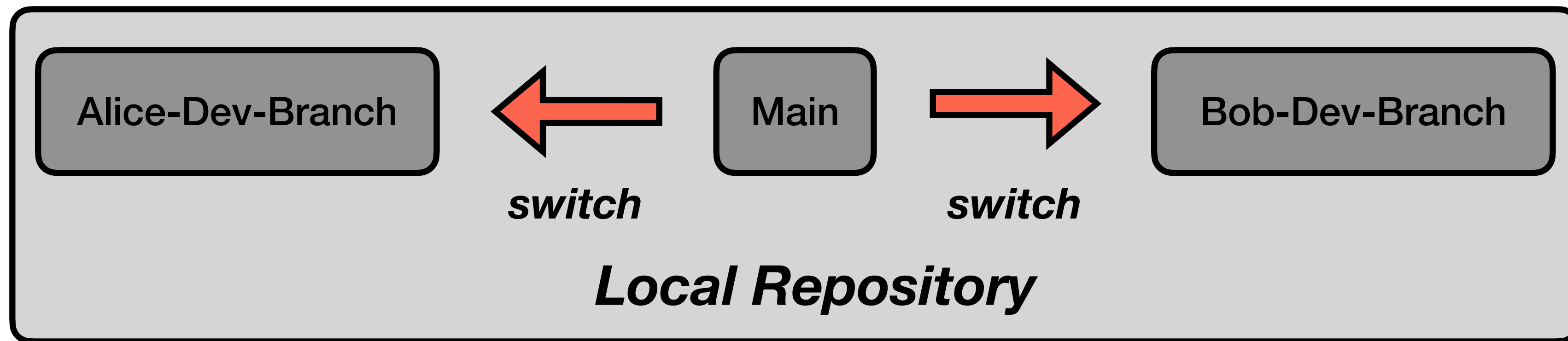
The diagram illustrates a conflict in a code repository. At the top, two gray boxes represent code snippets. The left box, associated with a blue developer icon, contains a comment '# Alice was here :)' on line 103. The right box, associated with a green developer icon, contains a comment '# Bob was here :)' on line 103. Both boxes show identical code for lines 104 and 105. Red arrows from both boxes point down to a purple box labeled 'Remote Repository', indicating that both changes are being pushed to the same central location, creating a conflict.

**Remote Repository**

**Branches Make Things Easier!**

***Each branch is a “separate local repository”***

*If you break something, it’s isolated to the current branch...*

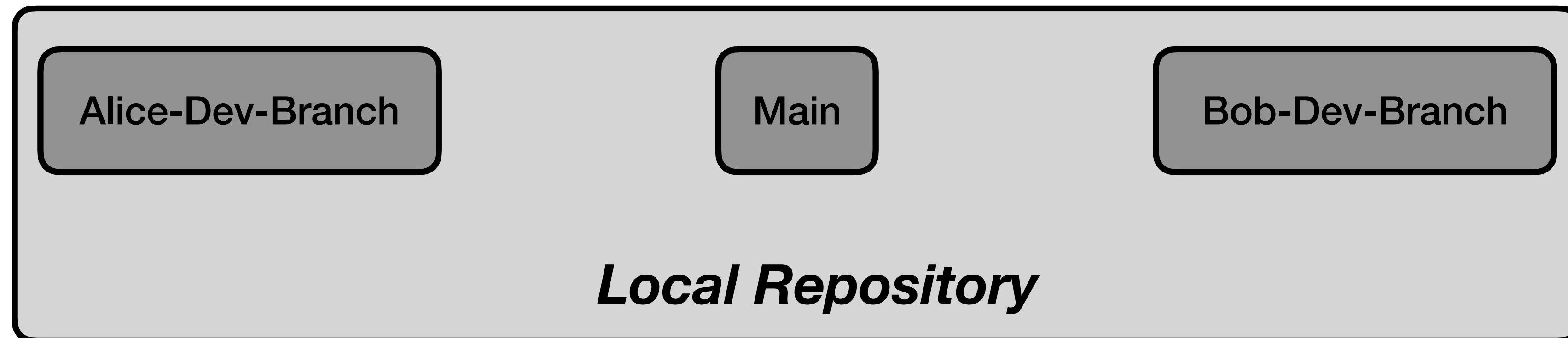
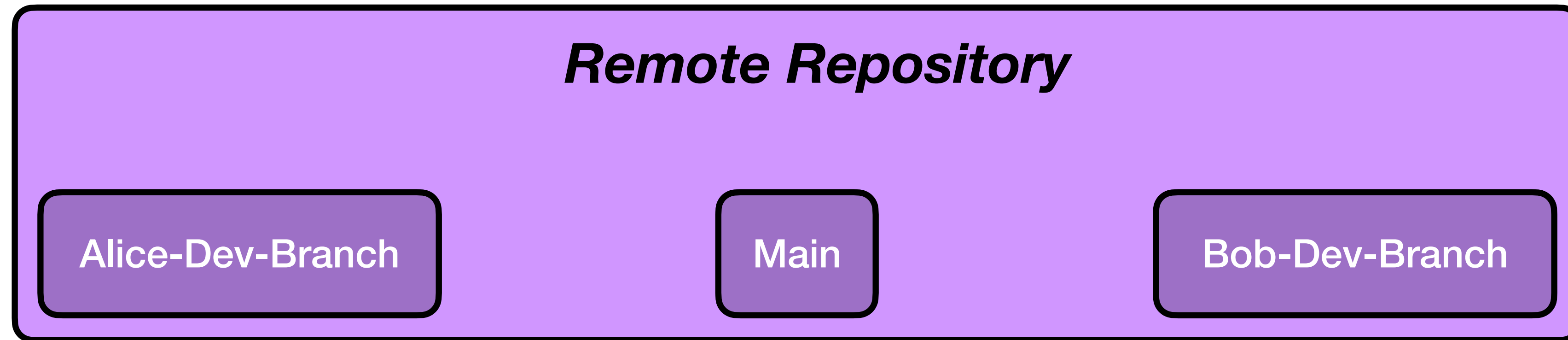


*It’s fairly easy to resolve conflicts between branches!*

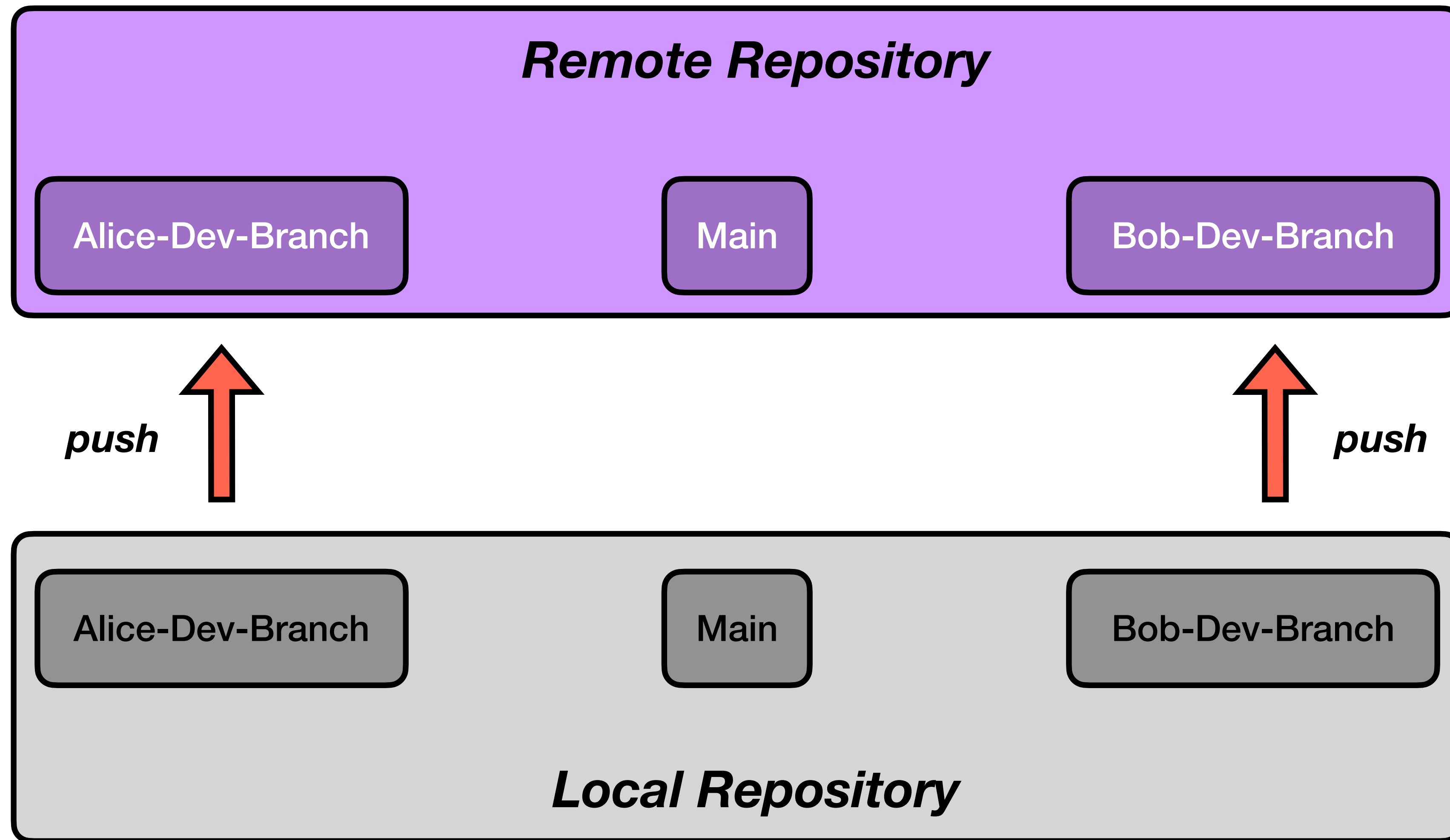
The branch you are currently coding is called the **working branch**



**Code development should flow like a pot of boiling water...**

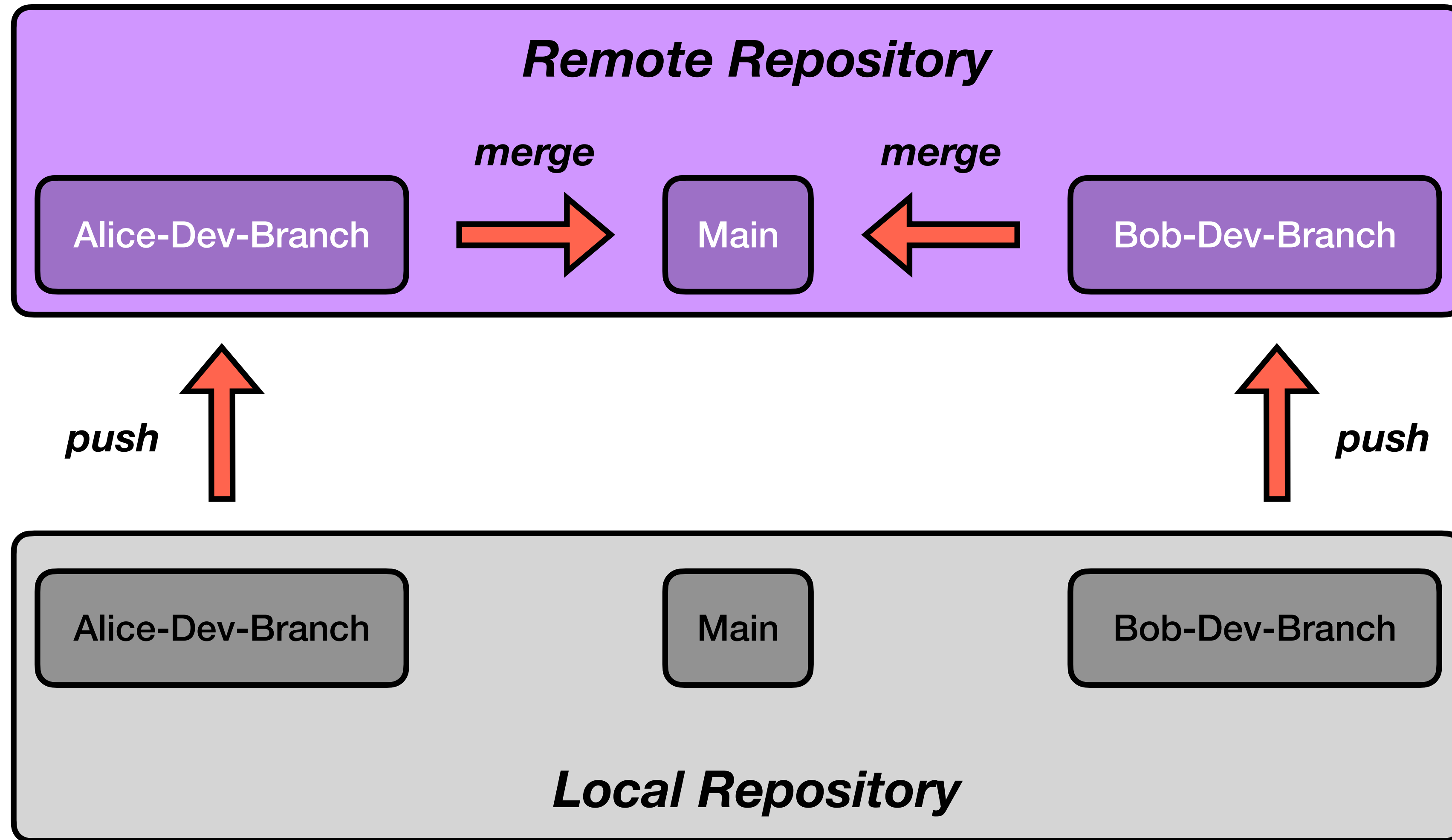


**Code development should flow like a pot of boiling water...**

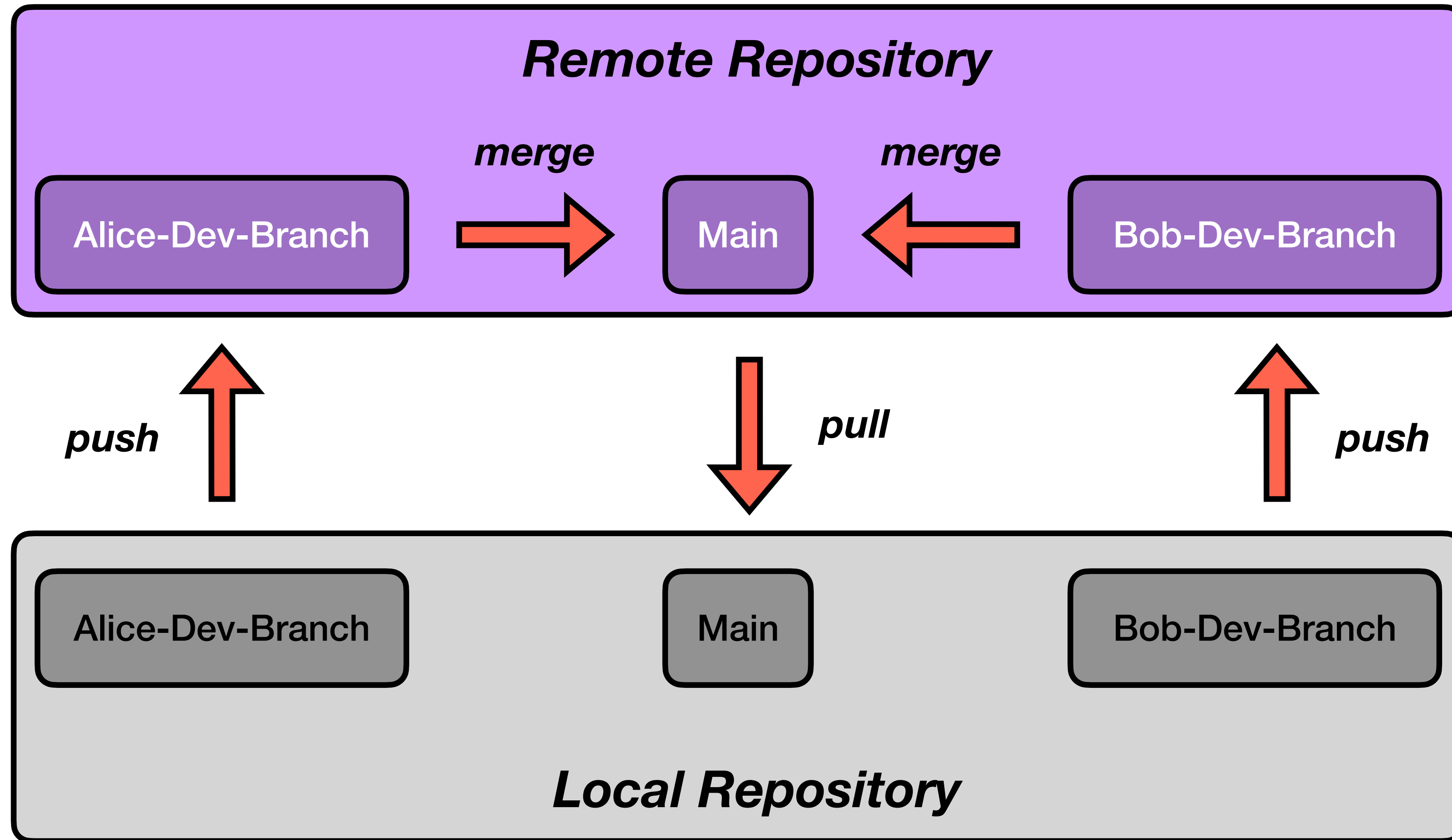




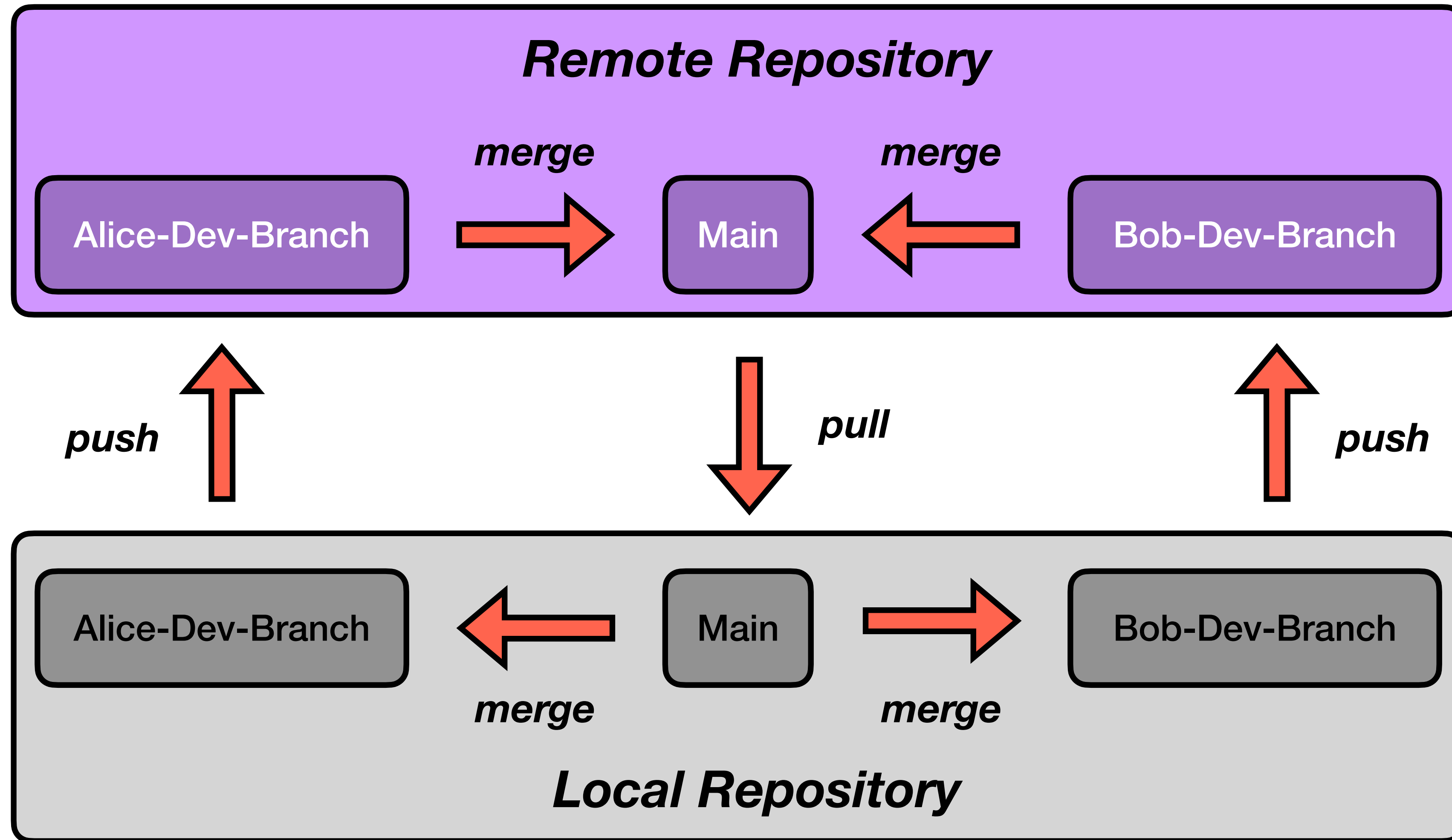
**Code development should flow like a pot of boiling water...**



**Code development should flow like a pot of boiling water...**



**Code development should flow like a pot of boiling water...**





## ***Creating & Listing Existing Branches***

> **git branch branch-name**

Creates a new branch on your local repository

> **git branch**

Lists all local branches (*current branch highlighted*)

```
● (base) practice_collaborative_coding % git branch autumn-dev
● (base) practice_collaborative_coding % git branch
    autumn-dev
* main
```

# Creating & Listing Existing Branches

> `git branch branch-name`

Creates a new branch on your local repository

> `git branch`

Lists all local branches (*current branch highlighted*)

```
● (base) practice_collaborative_coding % git branch autumn-dev
● (base) practice_collaborative_coding % git branch
  autumn-dev
* main
```

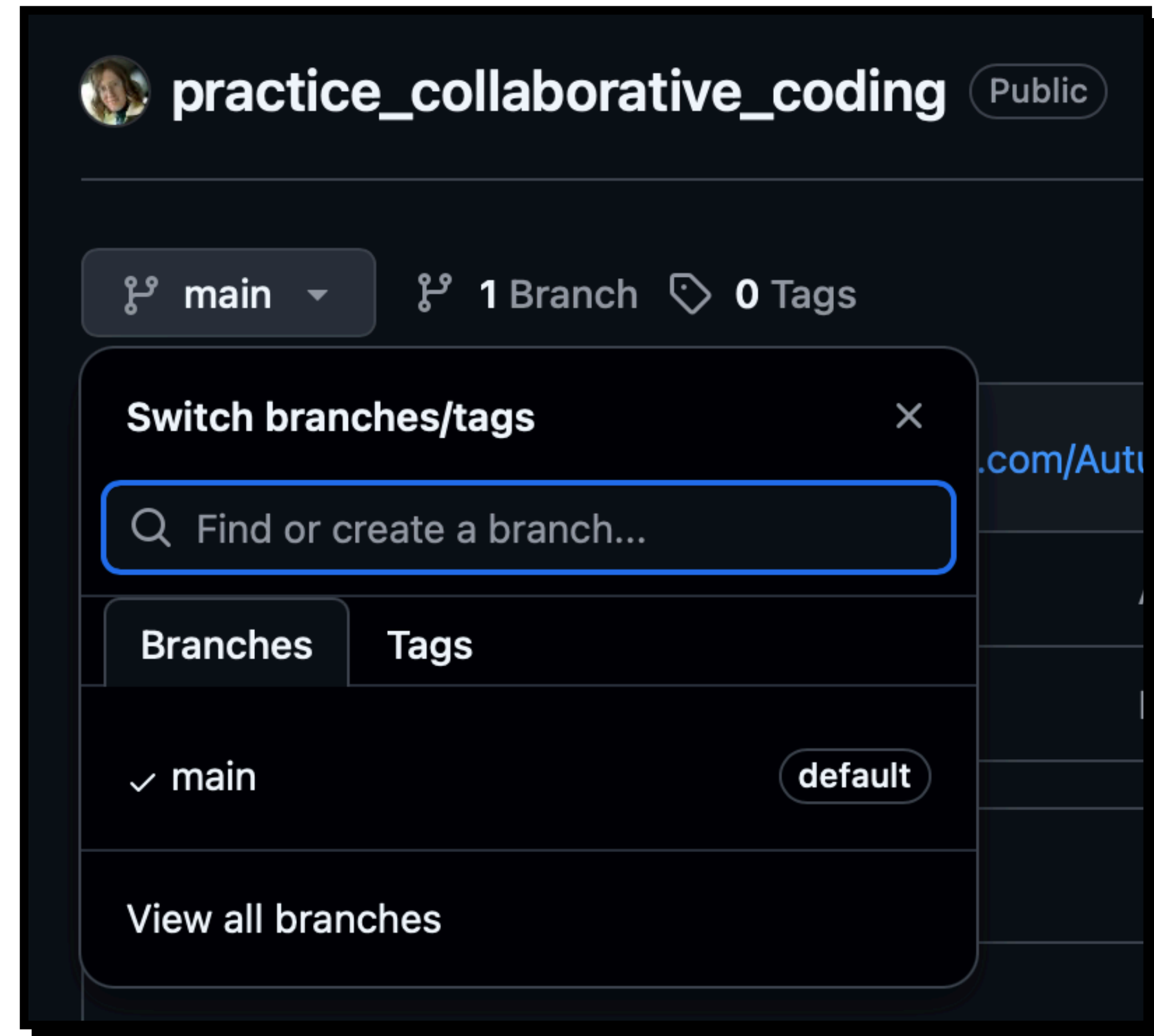
> `git switch branch-name`

Changes your current working branch

```
● (base) practice_collaborative_coding % git switch autumn-dev
Switched to branch 'autumn-dev'
● (base) practice_collaborative_coding % git branch
* autumn-dev
  main
```

# Creating & Listing Existing Branches

*So far, we only created a **new branch** on our **local repository**. If we look at GitHub, we can see that the only listed branch is main.*





# Creating & Listing Existing Branches

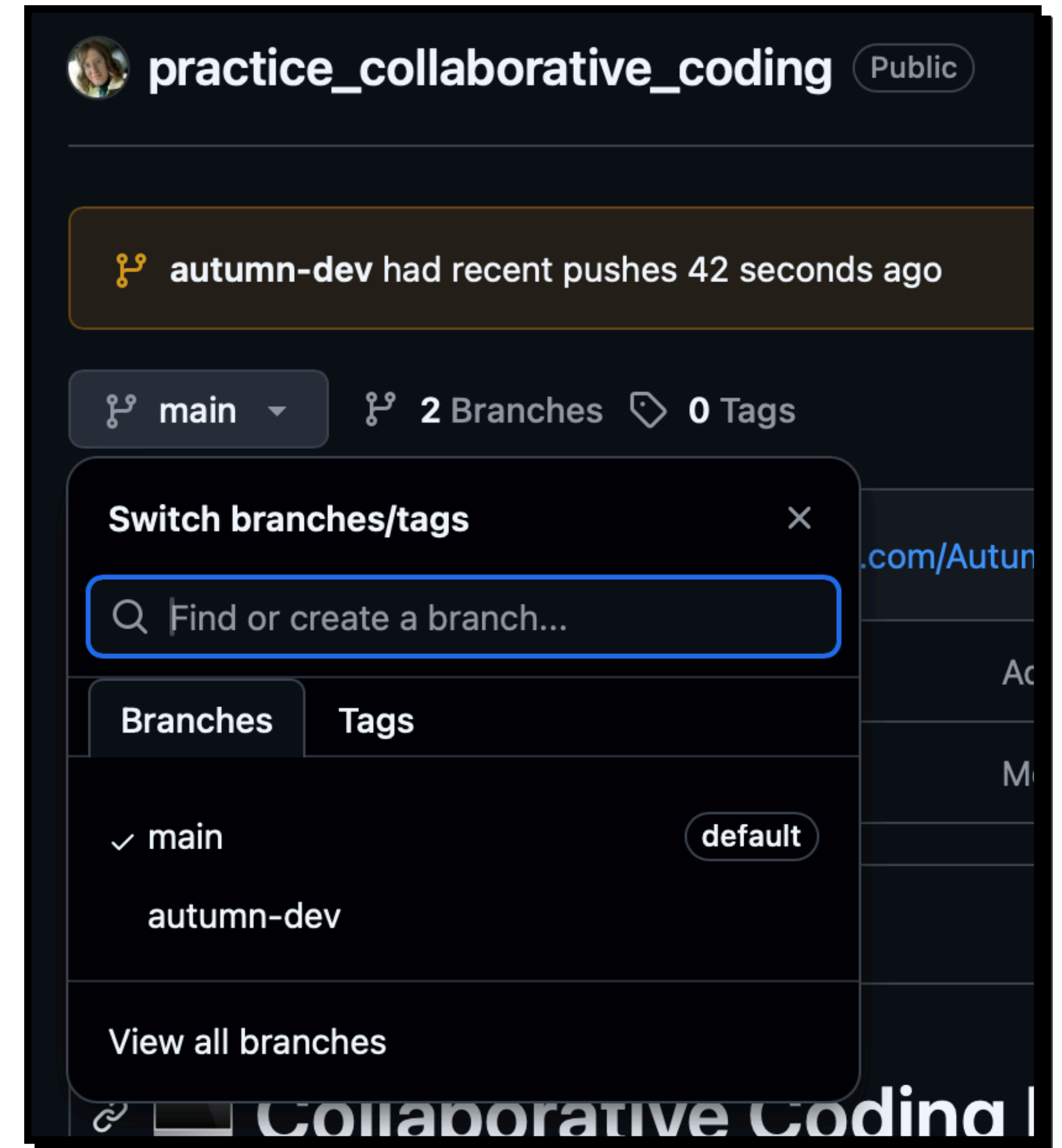
*Let's add, commit, and push our change!*

```
• (base) [redacted] practice_collaborative_coding % git add src/sample_code.py
• (base) [redacted] practice_collaborative_coding % git commit -m "Initial commit"
[autumn-dev a209c7d] Initial commit
 1 file changed, 8 insertions(+)
 create mode 100644 src/sample_code.py
• (base) aust8150@UCB-G0PQL4Q917 practice_collaborative_coding % git push origin autumn-dev
```

*When pushing to a new branch, you get this message...*

```
To https://github.com/Autumn10677/practice_collaborative_coding.git
* [new branch]      autumn-dev -> autumn-dev
```

*The GitHub repository has our branch!*



# Handling Merge Requests on GitHub

Autumn10677 / practice\_collaborative\_coding

Code

Issues

Pull requests

Agents

Actions

Projects

Wiki

Security and quality

Insights

Settings

Type / to search

practice\_collaborative\_coding

Public

Pin

Watch 0

autumn-dev had recent pushes 45 minutes ago

Compare & pull request

autumn-dev

2 Branches

0 Tags

Go to file

T

Add file

Code

This branch is 1 commit ahead of main

Contribute

Autumn10677

Initial commit

a209c7d · 45 minutes ago

14 Commits

|           |                                                            |                |
|-----------|------------------------------------------------------------|----------------|
| slides    | Added wk1 slides                                           | last week      |
| src       | Initial commit                                             | 45 minutes ago |
| README.md | Merge branch 'main' of https://github.com/Autumn10677/p... | last week      |

You can start a pull request using either of these options.

The “**Compare & pull request**” button usually only shows up for recent changes.



# Handling Merge Requests on GitHub

*Check the target / source branches, then create a pull request!*

## Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).



base: main

←

compare: autumn-dev

✓ **Able to merge.** These branches can be automatically merged.



**Add a title \***  
Initial commit

**Add a description**

WritePreview

H B I           

Add your description here...

 Markdown is supported  Paste, drop, or click to add files

Create pull request

 Remember, contributions to this repository should follow our [GitHub Community Guidelines](#).

**Reviewers**  
No reviews

**Assignees**  
No one—[assign yourself](#)

**Labels**  
None yet

**Projects**  
None yet

**Milestone**  
No milestone

**Development**  
Use [Closing keywords](#) in the description to automatically close issues

**Helpful resources**  
[GitHub Community Guidelines](#)

*Most of the options on the side here are not important for now. These can help you organize merge requests as a project grows in size.*

# Handling Merge Requests on GitHub

*Don't forget to update your local repository!*

```
● (base) [redacted] practice_collaborative_coding % git pull origin main
From https://github.com/Autumn10677/practice_collaborative_coding
 * branch          main          -> FETCH_HEAD
Updating e90ea0c..fd4390c
Fast-forward
 src/sample_code.py | 8 ++++++++
 1 file changed, 8 insertions(+)
 create mode 100644 src/sample_code.py
● (base) [redacted] practice_collaborative_coding % git switch autumn-dev
Switched to branch 'autumn-dev'
● (base) [redacted] practice_collaborative_coding % git merge main
```

*As long as no conflicts were detected, this should be relatively simple.*

**What if there is a conflict?**



# Merge Conflicts

## Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).

 base: main   compare: broken-branch  ✖ Can't automatically merge. Don't worry, you can still create the pull request.

## Adjusted values of 'np.arange' #3

Resolving conflicts between broken-branch and main and committing changes → broken-branch

1 conflicting file

 sample\_code.py  
src/sample\_code.py

src/sample\_code.py

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  <<<<<<< broken-branch (Current change)
5  # Changed end arguments of 'np.arange'
6  x_values = np.arange(0, 1, 0.1)
7  =====
8  # Plots the x^2 from -1 to 1
9  x_values = np.arange(-1, 1, 0.5)
10 >>>>>>> main (Incoming change)
11 y_values = x_values**2
12
13 plt.plot(x_values, y_values)
14 plt.show()
15 |
```

If conflicts are detected between your branch and main, you will need to tell GitHub or VS Code which changes to keep.

This is an example of a relatively minor conflict between two branches.



# Merge Conflicts

There is no good general rule here, what you choose will depend a lot on your project!

### Adjusted values of 'np.arange' #3

Resolving conflicts between `broken-branch` and `main` and committing changes → `broken-branch`

1 conflicting file

sample\_code.py

src/sample\_code.py

src/sample\_code.py

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 <<<<<< broken-branch (Current change)
5 # Changed end arguments of 'np.arange'
6 x_values = np.arange(0, 1, 0.1)
7 =====
8 # Plots the x^2 from -1 to 1
9 x_values = np.arange(-1, 1, 0.5)
10 >>>>>> main (Incoming change)
11 y_values = x_values**2
12
13 plt.plot(x_values, y_values)
14 plt.show()
15 |
```

## Accept current change

```
# Changed end arguments of 'np.arange'
x_values = np.arange(0, 1, 0.1)
y_values = x_values**2
```

## Accept incoming change

```
# Plots the x^2 from -1 to 1
x_values = np.arange(-1, 1, 0.5)
y_values = x_values**2
```

## Accept both changes

```
# Changed end arguments of 'np.arange'
x_values = np.arange(0, 1, 0.1)
# Plots the x^2 from -1 to 1
x_values = np.arange(-1, 1, 0.5)
y_values = x_values**2
```

How *should* code be written?

# ~~How *should* code be written?~~

*However you want! But, here are some suggestions...*



# ***Introduction to Best Practices***

## ***A Non-Exhaustive List of Suggestions:***

- Make sure code fails fast
- Avoid “magic numbers”
- Don’t write repetitive code
- Clear, concise comments where needed
- Write single-purpose functions
- Return results, don’t print them
- Handle errors internally and control behavior
- Use descriptive variable / function names
- Follow a style guide

*All of these are meant to make code easy to **test** & **read**!*



# ***Introduction to Best Practices***

## ***A Non-Exhaustive List of Suggestions:***

- Make sure code fails fast
  - Avoid “magic numbers”
  - Don’t write repetitive code
  - Clear, concise comments where needed
  - Write single-purpose functions
  - Return results, don’t print them
  - Handle errors internally and control behavior
  - Use descriptive variable / function names
- Follow a style guide

*All of these are meant to make code easy to **test** & **read**!*

*Let’s start with **style guides**!*

# ***flake8*: A Simple Checker for the PEP-8 Style Guide**

*This code would run, but we can improve readability!*

```
sample_code.py > ...  
1  import numpy as np  
2  import matplotlib.pyplot as plt  
3  
4  x_values = np.arange(0,10,1)  
5  y_values = x_values ** 2
```

# *flake8*: A Simple Checker for the PEP-8 Style Guide

*This code would run, but we can improve readability!*

```
sample_code.py > ...  
1  import numpy as np  
2  import matplotlib.pyplot as plt  
3  
4  x_values = np.arange(0,10,1)  
5  y_values = x_values ** 2
```

*Using **flake8**, we can get an easy-to-read list of style guide mistakes*

```
(base) % flake8 sample_code.py  
sample_code.py:2:1: F401 'matplotlib.pyplot as plt' imported but unused  
sample_code.py:4:23: E231 missing whitespace after ','  
sample_code.py:4:26: E231 missing whitespace after ','  
sample_code.py:5:25: W292 no newline at end of file
```



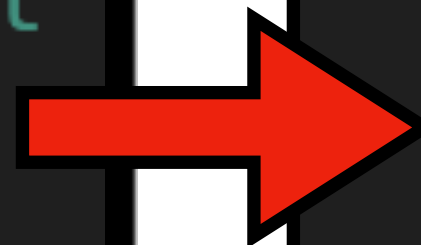
# flake8: A Simple Checker for the PEP-8 Style Guide

```
ⓧ (base) % flake8 sample_code.py
sample_code.py:2:1: F401 'matplotlib.pyplot as plt' imported but unused
sample_code.py:4:23: E231 missing whitespace after ','
sample_code.py:4:26: E231 missing whitespace after ','
sample_code.py:5:25: W292 no newline at end of file
```

Each entry has the following format...

***file\_name*** : ***Line #*** : ***Character #*** : ***Rule ID*** / ***Explanation***

```
🔗 sample_code.py > ...
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  x_values = np.arange(0,10,1)
5  y_values = x_values ** 2
```



```
🔗 sample_code.py > ...
1  import numpy as np
2
3  x_values = np.arange(0, 10, 1)
4  y_values = x_values ** 2
5
```

## Other Style Guides

*Black — Automatically fixes files for you!*

```
● (base) % black sample_code.py
reformatted sample_code.py

All done! ✨ 🍰 ✨
1 file reformatted.
```

*pylint — Conforms to PEP-8*

*pycodestyle — Also uses PEP-8*

*Ruff — Never used this one, but seems popular!*

# A Handy Cheat Sheet

git...

|               |                                                                          |
|---------------|--------------------------------------------------------------------------|
| ...clone      | First-time <b>download and setup</b> of local a repository               |
| ...fetch      | Informs the <b>local repository</b> about <b>remote branch names</b>     |
| ...add        | <b>Stages</b> a changed file for saving                                  |
| ...commit     | Create a <b>new save state</b> with all new changes                      |
| ...push       | <b>Update remote</b> repository <b>using local</b> changes               |
| ...pull       | <b>Update local</b> repository <b>using remote</b> changes               |
| ...branch     | <b>List</b> all <b>existing branches</b> on your <b>local</b> repository |
| ...branch xxx | <b>Create</b> a <b>new branch</b> on your <b>local</b> repository        |
| ...merge xxx  | <b>Merge</b> branch “xxx” into your <b>working branch</b>                |



# Next Time...

Unit Testing & GitHub Actions